PROJECT PHASE - II

BABY CARE MANAGEMENT APPLICATION

A Project Phase - II Report Submitted In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology in **Computer Science and Engineering**

by

CH. SHIVARAMAKRISHNA (22N31A0540) **A. SUMANTH** (22N31A0508) **G. RAKESH** (22N31A0563)

Under the esteemed guidance of **P. Andrews Himakiran** Associate Professor

Department of Computer Science and Engineering Malla Reddy College of Engineering & Technology
(Autonomous Institution- UGC, Govt. of India) (Affiliated to JNTUH, Hyderabad, Approved by AICTE, NBA & NAAC with 'A' Grade) Maisammaguda, Kompally, Dhulapally, Secunderabad – 500100 website: www.mrcet.ac.in

2023-2024

Malla Reddy College of Engineering & Technology

(Autonomous Institution- UGC, Govt. of India) (Affiliated to JNTUH, Hyderabad, Approved by AICTE, NBA & NAAC with 'A' Grade) Maisammaguda, Kompally, Dhulapally, Secunderabad – 500100 website: www.mrcet.ac.in

CERTIFICATE

This is to certify that this is the bonafide record of the project entitled "Baby Care Management Application", submitted by CH. SHIVARAMAKRISHNA (22N31A0540), A. SUMANTH (22N31A0508) and G. RAKESH (22N31A0563) of B.Tech in the partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering, Department of CSE during the year 2023-2024. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide	Project Coordinator	Head of the Department
P. Andrews Himakiran	P. Andrews Himakiran	Dr. G. Ravi
(Associate Professor)	(Associate Professor)	(Professor)

External Examiner

DECLARATION

We hereby declare that the Project Phase-II report titled "Baby Care Management Application" submitted to Malla Reddy College of Engineering and Technology (UGC Autonomous), affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a result of original research carried-out in this thesis. It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of a degree.

CH. SHIVARAMAKRISHNA (22N31A0540) **A. SUMANTH** (22N31A0508) **G. RAKESH** (22N31A0563)

ACKNOWLEDGEMENT

We feel honored to place our warm salutation to our college Malla Reddy College of Engineering and Technology (UGC-Autonomous) for giving us an opportunity to do this Project as part of our B.Tech Program. We are ever grateful to our Principal Dr. V. Murali Mohan who enabled us to have experience in engineering and gain profound technical knowledge.

We express our heartiest thanks to our HOD, Dr. G. Ravi for encouraging us in every aspect of our course and helping us realize our full potential.

We would like to thank our Project Guide Mr. P. A. HimaKiran for his regular guidance, suggestions, and constant encouragement. We are extremely grateful to our Project Coordinator Mr. P. A. HimaKiran for his continuous monitoring and unflinching co-operation throughout project work.

We would like to thank our Class Incharge Mr. N. Siva Kumar who in spite of being busy with his academic duties took time to guide and keep us on the correct path.

We would also like to thank all the faculty members and supporting staff of the Department of CSE and all other departments who have been helpful directly or indirectly in making our project a success.

We are extremely grateful to our parents for their blessings and prayers for the completion of our project that gave us strength to do our project.

With regards and gratitude,

CH. SHIVARAMAKRISHNA (22N31A0540) **A. SUMANTH** (22N31A0508) **G. RAKESH** (22N31A0563)

ABSTRACT

The parent-child management ecosystem has become increasingly complex in the digital age, requiring modern families to juggle physical health tracking, medical appointments, cognitive development, and digital media consumption. The "Baby Care Management Application" project presents a comprehensive, full-stack web-based platform designed to securely and intuitively manage a child's holistic lifestyle and developmental history. This application leverages cutting-edge web technologies, specifically the MERN stack (MongoDB, Express.js, React.js, Node.js), to provide a unified environment for multi-faceted child care.

Users interact with the application through an intuitive web interface tailored for three distinct roles: Parents, Children, and Healthcare Professionals (Doctors). The system employs robust state-based authentication algorithms (JWT) and family grouping logic to ensure strict data privacy and isolation of family records. Parents can continuously track medical histories, including hospital bills, medication regimes, vaccination schedules, and growth metrics (BMI, Height, Weight over time).

To enhance cognitive and behavioral development, the application features a controlled learning and Media Center. This module offers age-appropriate video content distributions strictly governed by parental controls. It utilizes a real-time screen-time tracking algorithm that monitors viewing duration and automatically locks the child out upon reaching the parent-defined daily limit.

Furthermore, the platform integrates a dedicated medical portal that bridges the gap between home care and professional healthcare. It allows for seamless appointment booking by parents and enables medical professionals to directly interface with a child's health profile, adding clinical feedback, digital prescriptions, and developmental suggestions. Incorporating robust data protection measures to comply with medical data handling standards, the

Baby Care application guarantees secure handling of sensitive information, ultimately offering a highly reliable, centralized ecosystem for the modern paradigm of smart parenting.

TABLE OF CONTENTS

S.NO	TITLE	PG.NO
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	
	1.2 PROBLEM STATEMENT	
	1.3 PURPOSE AND OBJECTIVES	
	1.4 PROPOSED SOLUTION OVERVIEW	
	1.5 SCOPE OF THE PROJECT	
	1.6 ORGANIZATION OF THE REPORT	
2	**LITERATURE REVIEW**	**5**
	2.1 EXISTING SYSTEMS	
	2.2 DRAWBACKS OF EXISTING SYSTEMS	
	2.3 ADVANTAGES OF PROPOSED SYSTEM	
3	**SYSTEM ANALYSIS**	**8**
	3.1 FEASIBILITY STUDY	
	3.2 SYSTEM ENVIRONMENT	
	3.3 HARDWARE & SOFTWARE REQUIREMENTS	
	3.4 TECHNOLOGIES USED	
4	**SYSTEM REQUIREMENT SPECIFICATION (SRS)**	**14**
	4.1 FUNCTIONAL REQUIREMENTS	
	4.2 NON-FUNCTIONAL REQUIREMENTS	
	4.3 USER CHARACTERISTICS	
5	**SYSTEM DESIGN**	**18**
	5.1 SYSTEM ARCHITECTURE	
	5.2 DATA FLOW DIAGRAMS (LEVEL 0, 1, 2)	
	5.3 ENTITY RELATIONSHIP DIAGRAM (ERD)	
	5.4 UML DIAGRAMS	
	5.5 DATABASE DESIGN & SCHEMA	

6	**METHODOLOGY & ALGORITHMS**	**28**
	6.1 SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)	
	6.2 SCREEN TIME MANAGEMENT ALGORITHM	
	6.3 ROLE-BASED ACCESS CONTROL (RBAC) LOGIC	
	6.4 CRYPTOGRAPHY AND SECURITY FLOW	
7	**IMPLEMENTATION**	**33**
	7.1 ENVIRONMENT SETUP	
	7.2 BACKEND IMPLEMENTATION	
	7.3 FRONTEND IMPLEMENTATION	
	7.4 SAMPLE CODE IMPLEMENTATIONS	
8	**SOFTWARE TESTING**	**42**
	8.1 INTRODUCTION TO TESTING	
	8.2 UNIT TESTING	
	8.3 INTEGRATION TESTING	
	8.4 SYSTEM TESTING	
	8.5 TEST CASES	
9	**RESULTS & OUTPUT SCREENS**	**46**
	9.1 DESCRIPTION OF UI/UX	
	9.2 PROJECT SCREENSHOTS	
10	**CONCLUSION & FUTURE SCOPE**	**52**
	10.1 CONCLUSION	
	10.2 FUTURE ENHANCEMENTS	
11	**BIBLIOGRAPHY**	**54**

---

1. INTRODUCTION

1.1 BACKGROUND

In the contemporary era, the paradigm of parenting has shifted significantly. With the advent of dual-income households and nuclear family structures, parents find themselves stretched thin across professional responsibilities and the demanding task of child-rearing. Keeping track of a child's developmental milestones, vaccination schedules, daily medications, and cognitive development activities requires meticulous organization, which is often lost in the chaos of daily life.

Simultaneously, children are exposed to digital media at an increasingly younger age. While digital content can be incredibly educational, unrestricted access poses severe risks to a child's psychological and physical well-being, including screen addiction, sleep deprivation, and exposure to inappropriate content. Furthermore, coordinating with pediatricians and maintaining a chronological, easily accessible medical history for a child is largely done through physical files, which are prone to damage and loss. There is a pressing need for a unified digital solution that addresses these distinct but interconnected aspects of modern child care.

1.2 PROBLEM STATEMENT

The primary problem modern parents face is the fragmentation of child care management tools. Health records are kept in physical files; appointments are tracked in generic calendar apps; educational tracking is done manually or through isolated apps; and media consumption is difficult to monitor accurately across devices. This fragmentation leads to:

1. **Loss of Medical History:** Missing physical files or inability to recall past medical issues accurately when visiting a doctor.
2. **Unregulated Screen Time:** Inability to effectively implement and enforce strict daily screen time limits on digital devices.
3. **Communication Gap with Doctors:** Difficulty in efficiently booking appointments and receiving digital, persistent feedback/prescriptions from pediatricians.
4. **Lack of Centralized Oversight:** The absence of a single "dashboard" that gives parents a holistic view of their child's health, schedule, and learning progress.

1.3 PURPOSE AND OBJECTIVES

The Baby Care Management Application is designed to solve these problems by providing a centralized web portal.

Key Objectives:

1. **Holistic Family Dashboard:** To create a secure, multi-tenant portal where a parent can manage profiles for multiple children from a single account.
2. **Digital Health Record (EHR) Integration:** To allow parents to upload hospital bills, log daily medicines, track vaccinations on a timeline, and plot physical growth metrics.
3. **Strict Media Regulation:** To build an interactive Media Center for children that is strictly governed by parent-defined time limits, utilizing a real-time tracking algorithm to prevent overconsumption.
4. **Dedicated Doctor Ecosystem:** To provide healthcare professionals with an isolated portal to view assigned patients, their medical history, manage appointments, and write persistent digital feedback and prescriptions directly into the child’s record.
5. **Role-Based Isolation:** To ensure rigorous security through Role-Based Access Control (RBAC), ensuring that children cannot bypass screen locks, and doctors can only see patients specifically assigned to them.

1.4 PROPOSED SOLUTION OVERVIEW

The proposed Baby Care Management System is a full-stack web application built on the MERN stack. It handles distinct user roles explicitly.

- **The Parent Role** has absolute administrative privileges over their family unit. They can add children, set PINs for child login, manage medical records, define media limits, and interact with doctors.
- **The Child Role** has a highly restricted, intuitive interface. Upon logging in with a localized PIN, they are presented with a gamified dashboard showing their daily tasks and the Media Center, which locks automatically when time expires.
- **The Doctor Role** features a professional clinical dashboard. They can receive appointment requests, review a child’s complete uploaded medical history (to make informed decisions), and append permanent clinical feedback.

1.5 SCOPE OF THE PROJECT

The scope of this project involves designing, coding, testing, and deploying the complete application.

- **Frontend Scope:** Development of responsive, mobile-friendly User Interfaces using React.js and Vite. Creating separate thematic UI layouts for Parents, Children, and Doctors.
- **Backend Scope:** Development of RESTful APIs using Node.js and Express.js to handle user authentication, data manipulation, file uploads (for medical bills), and business logic for time tracking.
- **Database Scope:** Designing optimized NoSQL document schemas in MongoDB to efficiently manage deeply nested relationships (e.g., Parent->Child->MedicalRecords->Feedback).
- **Security Scope:** Implementing JSON Web Tokens (JWT) for session management, bcrypt for password hashing, and API route guards.

1.6 ORGANIZATION OF THE REPORT

The report is structured into 11 chapters. Chapter 1 introduces the project. Chapter 2 reviews existing literature and systems. Chapter 3 details system analysis and requirements. Chapter 4 provides the Software Requirement Specification (SRS). Chapter 5 delves deep into system design, including architectures and UML/DFD diagrams. Chapter 6 explains the methodology and core algorithms. Chapter 7 covers technical implementation. Chapter 8 discusses testing strategies. Chapter 9 presents the output screens. Finally, Chapter 10 concludes the report with future recommendations.

---

2. LITERATURE REVIEW

2.1 EXISTING SYSTEMS

Before developing the Baby Care Management Application, an extensive review of existing applications in the market was conducted.

1. **Standalone Health Trackers (e.g., Baby Tracker, Glow Baby):** These apps are excellent for tracking feeding times, diaper changes, and sleep schedules for infants. However, they lack long-term chronic medical history tracking and do not extend into early childhood education or media management.
2. **Parental Control Apps (e.g., Google Family Link, Qustodio):** These applications are highly effective at device-level locking and screen time management. However, they are isolated utility apps. They do not offer educational content, nor do they integrate with any health or scheduling facets of the child's life.
3. **Telehealth Platforms (e.g., Practo):** Useful for booking appointments with pediatricians, but they provide a generic platform that is not tailored to the continuous, everyday development metrics of a child.

2.2 DRAWBACKS OF EXISTING SYSTEMS

Based on the review, several critical drawbacks were identified across current market offerings:

- **Severe Fragmentation:** Parents must install, maintain, and secure 4 to 5 different applications to manage health, screen time, education, and medical appointments.
- **Lack of Role-Based Interfaces:** Most apps assume the parent is the only user. They do not provide a safe, isolated "kid's mode" within the same ecosystem where the child can interact safely.
- **Data Silos:** Because systems are distinct, a doctor cannot easily see the child's daily habits, and a parent cannot automatically correlate behavioral issues to health records.
- **Generic Medical Portals:** Telehealth portals treat pediatric appointments the same as adult appointments, failing to account for specific child metrics like growth percentiles and vaccination chronologies.

2.3 ADVANTAGES OF PROPOSED SYSTEM

The Baby Care Management Application introduces a paradigm shift by centralizing these systems.

- **All-in-One Dashboard:** Combines Medical History, Daily Task Management, Screen Time Control, and Doctor Appointments into a single unified interface.
- **Dynamic Role Interfaces:** A single web URL serves entirely different, purpose-built user interfaces depending on whether a Parent, Child, or Doctor logs in.
- **Tightly Coupled Security:** Medical records are cryptographically tied to the Parent's account, but selectively accessible by the specific Doctor assigned to an appointment, ensuring HIPAA-like privacy models.
- **Automated Enforcement:** The system doesn't just "report" screen time; it actively intervenes. The React frontend constantly polls the Node backend, and upon time expiration, forcefully unmounts the video player and renders an un-bypassable lock screen.

---

3. SYSTEM ANALYSIS

3.1 FEASIBILITY STUDY

A feasibility study was conducted to ensure the project is viable, technically possible, and economically justifiable.

1. Technical Feasibility: The project leverages the MERN stack. Node.js is highly capable of handling the concurrent asynchronous requests required for real-time screen time polling. React.js is ideal for building the dynamic, state-heavy interfaces required for the dashboards. MongoDB's flexible schema is perfectly suited for storing varied medical documents and unstructured clinical feedback. Thus, the system is technically highly feasible.

2. Economic Feasibility: The project relies entirely on open-source technologies (React, Node, Express, MongoDB Community). The development environment (VS Code) is free. Deployment can be achieved using free tiers of cloud platforms (Render, Vercel, MongoDB Atlas) during the prototype phase. Therefore, the project requires zero licensing costs and is highly economically feasible.

3. Operational Feasibility: The operational success depends on user adoption. The interface is specifically designed with non-technical parents in mind, featuring large buttons, clear typography, and guided forms. For children, the interface is heavily visual. For doctors, it models standard clinical software. This user-centric design guarantees high operational feasibility.

4. Schedule Feasibility: The project scope was broken down into manageable sprints using Agile methodology. With clear modular divisions (Auth, Dashboard, Medical, Media, Doctor), the project fits comfortably within the academic semester timeframe.

3.2 SYSTEM ENVIRONMENT

The system operates in a standard Client-Server architecture over the HTTP/HTTPS protocol. The server is responsible for all heavy lifting, data persistence, and cryptographic verification, ensuring that the client device (laptop, tablet, or smartphone) requires minimal computational power.

3.3 HARDWARE & SOFTWARE REQUIREMENTS

Hardware Requirements (Client Side):

- Processor: Any device with a modern dual-core processor (x86 or ARM).
- Ram: Minimum 2GB (Standard mobile device capability).
- Screen Resolution: Responsive design supports from 320x480 (Mobile) to 1920x1080 (Desktop).
- Network: Stable internet connection (3G/4G/Wi-Fi).

Hardware Requirements (Server/Development Side):

- Processor: Intel Core i5 / AMD Ryzen 5 or higher.

- RAM: 8 GB minimum (16 GB Recommended for running local DB, Node server, and React compiler simultaneously).
- Disk Space: Minimum 20 GB of free SSD space.

Software Requirements:

- Operating System: Windows 10/11, macOS Monterey+, Ubuntu 20.04+.
- Web Browser: Google Chrome (version 90+), Mozilla Firefox, or Safari (Requires HTML5 Video support).
- Development Environment: Node.js (v18.x LTS), npm (v9.x).
- Database: MongoDB Community Server (v6.x) or MongoDB Atlas Cloud.
- IDE: Visual Studio Code with ESLint and Prettier extensions.

3.4 TECHNOLOGIES USED

1. MongoDB (Database): MongoDB is a source-available cross-platform document-oriented database program. Classified as a NoSQL database program, MongoDB uses JSON-like documents with optional schemas. It was chosen because health records are highly variable (a vaccination record looks very different from a hospital bill record). A schema-less design allows rapid iteration.

2. Express.js (Backend Framework): Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It facilitates the rapid creation of API endpoints and handles HTTP request/response lifecycles, body parsing, and route middleware natively.

3. React.js & Vite (Frontend): React is a declarative, efficient, and flexible JavaScript library for building user interfaces. It lets developers compose complex UIs from small and isolated pieces of code called "components." We utilized **Vite** as our build tool. Vite provides a significantly faster and leaner development experience for modern web projects compared to Create React App (CRA), utilizing native ES modules.

4. Node.js (Runtime Environment): Node.js is an open-source, cross-platform, back-end JavaScript runtime environment that runs on the V8 engine and executes JavaScript code outside a web browser. Its event-driven, non-blocking I/O model makes it lightweight and highly efficient for data-intensive real-time applications.

5. JSON Web Tokens (JWT): JWT is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed using a secret key.

---

4. SYSTEM REQUIREMENT SPECIFICATION (SRS)

4.1 FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system should do.

FR1: User Authentication and Onboarding

- **FR1.1:** The system shall allow Users to register as a "Parent" or a "Doctor".
- **FR1.2:** The system shall encrypt all passwords using bcrypt before saving to the database.
- **FR1.3:** The system shall issue a JWT upon successful login, valid for session maintenance.

FR2: Parent Module Functionalities

- **FR2.1:** Overarching Family Dashboard to view summaries of all children.
- **FR2.2:** Ability to construct a "Child Profile" (Name, DOB, Gender, Blood Group).
- **FR2.3:** Ability to set a distinct 4-digit PIN for child login.
- **FR2.4:** Ability to set daily screen time limits (in minutes) for each child independently.
- **FR2.5:** Ability to upload images/PDFs of hospital bills and medical prescriptions.

- **FR2.6:** Ability to log daily medications and view them on a calendar list.

FR3: Child Module Functionalities

- **FR3.1:** The system shall provide a PIN-based login interface matching the Child's name.
- **FR3.2:** The system shall display a Media Center with categorized video content.
- **FR3.3:** The system shall track the duration of video playback continuously.
- **FR3.4:** The system shall automatically interrupt playback and render an overriding lock screen if the consumed time matches or exceeds the parent's limit.

FR4: Doctor Module Functionalities

- **FR4.1:** The system shall provide a dashboard to view pending and approved appointments.
- **FR4.2:** The system shall allow the Doctor to access the specific medical history (vaccines, bills) of a child assigned to them.
- **FR4.3:** The system shall provide a form for the Doctor to submit permanent clinical feedback and digital prescriptions to the child's profile.

4.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe how the system works.

NFR1: Security and Data Privacy

- **Cross-Site Scripting (XSS) Prevention:** The React frontend automatically escapes string variables in views.
- **Token Security:** JWTs must be maintained securely on the client side, and every protected backend route must execute an `authMiddleware` verification step.
- **Data Isolation:** A parent API request must intrinsically filter by `owner_id` to prevent accessing another family's data, even if brute-forced.

NFR2: Performance and Response Time

- API responses for standard database queries (e.g., loading dashboard data) should resolve in under 500 milliseconds under standard load protocols.
- The React application should complete its initial DOM render rapidly, utilizing code-splitting where necessary.

NFR3: Reliability and Availability

- The system should maintain stable connections during child media playback to prevent false positives in screen time tracking.

NFR4: Usability

- The design must follow responsive web design principles to work flawlessly on smartphones, as parents primarily use mobile devices for tracking applications.

4.3 USER CHARACTERISTICS

1. **Parents:** General users with basic technical literacy. They expect a clean, intuitive, and highly responsive interface with clear error messaging.
2. **Children:** Users ranging from toddlers to early teens. They do not possess technical literacy. The interface must be visually driven, with large interactive elements, icons, and minimal text reading requirements.
3. **Doctors:** Professional users. They require a highly organized, data-dense interface that mimics standard clinical software for rapid review of chronological medical data.

---

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The system is built on a robust 3-Tier Architecture model:

1. **Presentation Tier (Client-Side):**
    - Built with React, HTML5, and CSS3.
    - Responsible for rendering the UI components based on the user's role (Parent Dashboard, Child Media Center, Doctor Portal).
    - Manages local state (e.g., active tab, form inputs) and global state (authentication status).
    - Communicates with the Logic Tier via Axios/Fetch API using JSON over HTTP.
2. **Logic/Application Tier (Server-Side):**
    - Built with Node.js and Express.js.
    - Acts as the central nervous system.
    - Contains route definitions (e.g., `/api/users`, `/api/children`, `/api/medical`).
    - Executes middleware functions (Authentication verification, Error Handling).
    - Contains Controller functions executing the core business logic (e.g., calculating screen time remaining, orchestrating Database calls).
3. **Data Tier (Database):**
    - Built with MongoDB.
    - Receives structured queries (via Mongoose ODM) from the Application Tier.
    - Persists all relational and non-relational data securely.

5.2 DATA FLOW DIAGRAMS (LEVEL 0, 1, 2)

DFD Level 0 (Context Diagram): In the Level 0 DFD, the entire "Baby Care Management System" is represented as a single central process.

- Input entities: Parent, Child, Doctor.
- The Parent inputs registration details, child profiles, medical uploads, and appointment requests into the System. The System outputs dashboard metrics and doctor feedback back to the Parent.
- The Child inputs PIN login and media requests. The System outputs media content or lock screens.
- The Doctor inputs clinical feedback and appointment confirmations. The System outputs patient medical records and appointment requests to the Doctor.

DFD Level 1: The system is broken down into major sub-processes: 1.0 Authentication Management 2.0 Profile & Family Management 3.0 Medical Records Management 4.0 Screen Time & Media Control 5.0 Appointment & Feedback Management

(Note: Data stores such as UserDB, ChildDB, and MedicalDB are interacted with dynamically by these sub-processes)

DFD Level 2 (Focus: Screen Time Module): Breakdown of Process 4.0: 4.1 Validate Child Session Token against backend. 4.2 Retrieve `dailyTimeLimit` and `consumedTime` from ChildDB. 4.3 Decision Module: Compare values. If `consumedTime` >= `dailyTimeLimit`, trigger 4.4 (Render Lock Screen). 4.5 Trigger playback. Start foreground Web API Timer. 4.6 Ping backend (Process 4.6 Update Time API) every 60 seconds with increment data. 4.7 Save updated `consumedTime` to ChildDB. Repeat from 4.2.

5.3 ENTITY RELATIONSHIP DIAGRAM (ERD)

The database structure is designed around critical entities.

- **USER Entity:** `_id` (PK), `name`, `email`, `password`, `role`.
- **CHILD Entity:** `_id` (PK), `parentId` (FK to User), `name`, `dob`, `pinCode`, `screenTimeLimit`.
- **MEDICAL_RECORD Entity:** `_id` (PK), `childId` (FK to Child), `recordType` (Enum: Vaccine, Bill, Medicine), `notes`, `fileUrl`, `date`.
- **APPOINTMENT Entity:** `_id` (PK), `parentId` (FK to User), `doctorId` (FK to User), `childId` (FK to Child), `dateTime`, `status`, `reason`.
- **FEEDBACK/PRESCRIPTION Entity:** `_id` (PK), `doctorId` (FK to User), `childId` (FK to Child), `diagnosis`, `prescriptionNotes`, `timestamp`.

5.4 UML DIAGRAMS

5.4.1 CLASS DIAGRAM

The class diagram maps directly to our Mongoose Data Models. It illustrates the attributes, operations, and relationships among the system classes.

```
[ User Class ]
- _id: ObjectId
- name: String
- email: String
- password: String (Hashed)
- role: Enum ['Parent', 'Doctor']
+ login()
+ register()
+ generateToken()

       ^ (1)
       |
       | Has Many (*)
       v

[ Child Class ]
- _id: ObjectId
- parentId: ObjectId (Ref: User)
- name: String
- dob: Date
- pinCode: String
- dailyScreenTimeLimit: Number
- consumedScreenTime: Number
+ addMedicalRecord()
+ checkTimeLimit()

       ^ (1)
       |
       | Has Many (*)
       v

[ MedicalRecord Class ]
- _id: ObjectId
- childId: ObjectId (Ref: Child)
- documentSubtype: String
- uploadUrl: String
```

```
- details: String
- recordDate: Date
+ fetchRecords()
+ deleteRecord()
```

5.4.2 USE CASE DIAGRAM

A Use Case diagram models the dynamic behavior of the system by describing interactions between actors and the system.

- **Actor: Parent** -> Use Cases: {Register Account}, {Login}, {Create Child Profile}, {Set PIN & Limits}, {Upload Medical Bills}, {Log Vaccines}, {Search Doctor}, {Book Appointment}, {View Feedback}.
- **Actor: Child** -> Use Cases: {Login via PIN}, {Select Video}, {Watch Video}, {Trigger Screen Lock (System enforced)}.
- **Actor: Doctor** -> Use Cases: {Login}, {View Appointments Dashboard}, {Accept/Reject Appointment}, {View Assigned Child Medical History}, {Type & Submit Prescriptions}.

5.4.3 SEQUENCE DIAGRAM: SCREEN TIME ENFORCEMENT

This maps the chronological flow of messages for the most complex feature.

1. **Child (Frontend)** clicks "Watch Video".
2. **React App** sends `GET /api/child/:id/screentime` with Bearer Token.
3. **Express Node Backend** receives request, runs `authMiddleware`.
4. **Node Backend** queries **MongoDB** for Child document.
5. **MongoDB** returns `{ dailyLimit: 120, consumedTime: 60 }`.
6. **Node Backend** evaluates logic (60 < 120). Sends `HTTP 200 OK` to React.
7. **React Component** mounts Video Player.
8. (Loop begins every 1 minute) -> **React App** sends `PUT /api/child/:id/incrementTime` payload `{ amount: 1 }`.
9. **Node Backend** increments value in **MongoDB**.
10. If value hits limit, Backend responds `HTTP 403 Forbidden`.
11. **React App** catches 403, triggers `setLocked(true)`, unmounts Video Player.

5.4.4 ACTIVITY DIAGRAM: DOCTOR FEEDBACK

The activity diagram models the procedural flow of actions.

[Start] -> [Authenticate as Doctor] -> [Redirect to Doctor Dashboard] [View Appointment List] -> [Select Pending Appointment] [View Patient Details] -> [System fetches Medical Records] -> [Render History to Screen] [Doctor Analyzes History] -> [Doctor inputs Diagnosis & Prescription into Form] [Submit Button Clicked] -> [Backend Validation] -> (If Error) -> [Show Toast Error] -> [Return to Form] (If Success) -> [Save Feedback to DB] -> [Update Appointment Status to 'Completed'] -> [End Flow]

5.5 DATABASE DESIGN & SCHEMA

The project uses strict Mongoose Schemas to enforce data integrity in MongoDB. Example structure for a Medical Record:

- `owner` : Reference to User object (ensures security via parent lookup).
- `child` : Reference to Child object.
- `recordType` : String (Restricted via enum).
- `metrics` : Nested Object containing flexible data (e.g., `height`, `weight`, `vaccineName`, `billAmount`).

---

6. METHODOLOGY & ALGORITHMS

6.1 SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

The project was developed using the **Agile Software Development Methodology**. Unlike traditional Waterfall models, Agile promotes adaptive planning, evolutionary development, early delivery, and continual improvement.

1. **Sprint 1 (Requirements & Architecture):** Finalizing the tech stack (MERN), designing database schemas, defining API contracts.
2. **Sprint 2 (Authentication & Parent Dashboard):** Implementing JWT login, Role mapping. Building the Parent UI to perform CRUD operations on Child Profiles.
3. **Sprint 3 (Medical Tracker & File Handling):** Designing the schemas for tracking diverse medical records and building the UI to visualize this data (charts, tables).
4. **Sprint 4 (Media & Screen Time Core logic):** Developing the backend algorithms to track time increments and the React components to respond to limits.
5. **Sprint 5 (Doctor Portal Integration):** Connecting the ecosystems by enabling appointment logic and feedback forms.
6. **Sprint 6 (Testing & Polish):** Debugging edge cases, refining CSS for responsiveness.

6.2 SCREEN TIME MANAGEMENT ALGORITHM

The problem of accurately tracking time in a stateless HTTP environment is handled by a "pulse" algorithm.

Algorithm Definition:

```
Step 1: Initialize media component on Client.
Step 2: Fetch Initial State (limit L, consumed C) from Server.
Step 3: IF (C >= L) THEN
           Return Render_Lock_Screen()
        ELSE
           Return Render_Video_Player()
Step 4: Start JavaScript setInterval() timer on Client (Interval = X seconds).
Step 5: ON timer tick:
           Send non-blocking asynchronous request to Server API (/increment).
           Server increments C in database by X.
           Server evaluates (New C >= L).
           IF (true) THEN
               Server responds with { locked: true }
               Client catches response, clears interval, executes Render_Lock_Screen()
```

This pulse mechanism ensures that even if the client closes the browser abruptly, the server has accurately tracked the consumption up to the last X seconds.

6.3 ROLE-BASED ACCESS CONTROL (RBAC) LOGIC

To prevent a child from modifying their own screen time, or a parent from writing their own doctor prescriptions, RBAC is implemented both on the Client and Server.

Server-Side Logic: Custom Middleware `authorize(roles...)` wraps sensitive routes.

```
const authorize = (...roles) => {
  return (req, res, next) => {
```

```
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: "Role unauthorized to access this route" });
    }
    next();
  };
};
// Route declaration example:
router.post('/feedback', protect, authorize('Doctor'), addFeedback);
```

Client-Side Logic: React Router employs a Higher-Order Component (HOC) called `<ProtectedRoute>`. It reads the role from the stored Redux state or decoded Token. If the role does not match the prop `allowedRole`, it executes a `<Navigate to="/unauthorized" />`.

6.4 CRYPTOGRAPHY AND SECURITY FLOW

Data at Rest and Data in Transit are secured:

- **Hashing:** When a user registers, the plain-text password is intercepted by a Mongoose `pre-save` hook. `bcrypt.genSalt(10)` generates a random salt, which is fed into `bcrypt.hash()` alongside the password. The resulting cryptographic hash is saved. Password verification during login relies on `bcrypt.compare()`.
- **Stateless Tokens:** Upon authentication, Node.js signs a payload containing the User ID using symmetric HMAC SHA256 cryptographic algorithm (`jsonwebtoken` library) anchored by a secret key stored only in the server's `.env` repository.

---

7. IMPLEMENTATION

7.1 ENVIRONMENT SETUP

The development environment utilized modular architecture.

- A root directory containing server and client subdirectories.
- `package.json` configurations utilize concurrent packages allowing both React and Node servers to spin up simultaneously with hot-reloading (Nodemon and Vite HMR).

7.2 BACKEND IMPLEMENTATION

Backend implementation focuses on the `REST` architectural style.

- **Controllers:** Separated into `authController`, `childController`, `medicalController`, `doctorController`. Controllers parse requests, interact with Mongoose models, and return structured JSON responses.
- **Models:** Defined using Mongoose. Schemas include strict type validation and required constraints to prevent corrupted database entries.
- **Routes:** Modularized route handlers appended to the main Express app using `app.use('/api/...', ...Routes)`.

7.3 FRONTEND IMPLEMENTATION

- **Component Architecture:** React development was broken down into generic, reusable components (e.g., `<Button>`, `<Card>`, `<Modal>`) and specific Page components (`<ParentDashboard>`, `<DoctorPortal>`).

- **State Management:** React Context API or complex component state (`useState` , `useReducer`) is utilized to manage modal visibilities, form payloads, and fetched data.
- **Routing:** `<BrowserRouter>` from `react-router-dom` v6 orchestrates the navigation without page reloads, providing a Single Page Application (SPA) experience.

7.4 SAMPLE CODE IMPLEMENTATIONS

Code Snippet 1: User Model Schema (MongoDB/Mongoose)

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['Parent', 'Doctor', 'Admin'], default: 'Parent' },
  createdAt: { type: Date, default: Date.now }
});

// Middleware to hash passwords before saving
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Method to verify password
userSchema.methods.matchPassword = async function(enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

Code Snippet 2: JWT Authentication Middleware (Express)

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');
      next();
    } catch (error) {
      console.error(error);
      res.status(401).json({ message: 'Not authorized, token failed' });
    }
```

```
  }
  if (!token) {
    res.status(401).json({ message: 'Not authorized, no token present' });
  }
};
module.exports = protect;
```

Code Snippet 3: React Frontend Component - Screen Time Verification Flow

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

const MediaPlayer = ({ childId }) => {
  const [locked, setLocked] = useState(false);
  const [timeRemaining, setTimeRemaining] = useState(0);

  useEffect(() => {
    // Initial Verification Check
    const checkLimits = async () => {
      try {
        const { data } = await axios.get(`/api/child/${childId}/screentime`);
        if (data.consumed >= data.limit) {
          setLocked(true);
        } else {
          setTimeRemaining(data.limit - data.consumed);
        }
      } catch (err) {
         console.error("Verification failed", err);
      }
    };
    checkLimits();

    // Pulse Timer to interact with backend
    let interval;
    if (!locked) {
      interval = setInterval(async () => {
         try {
            const { data } = await axios.put(`/api/child/${childId}/incrementTime`);
            if (data.isLocked) {
               setLocked(true);
               clearInterval(interval);
            }
         } catch (error) {
            console.error("Pulse failed");
         }
      }, 60000); // 1 minute pulse
    }

    return () => clearInterval(interval);
  }, [childId, locked]);
```

```
  if (locked) {
    return <div className="lock-screen"><h1>Time's Up!</h1><p>Ask a parent to unlock more
time.</p></div>;
  }

  return (
    <div className="video-player">
      <video src="/content/educational.mp4" controls autoPlay />
    </div>
  );
};

export default MediaPlayer;
```

---

8. SOFTWARE TESTING

8.1 INTRODUCTION TO TESTING

Testing is a critical phase of the SDLC, guaranteeing the quality, reliability, and security of the application. Given the sensitive nature of medical data and the strict enforcement rules required for parental controls, rigorous testing methodologies were implemented.

8.2 UNIT TESTING

Unit testing involves verifying the smallest testable parts of the code individually.

- Backend functions like `calculateAge()` or `comparePasswords()` were tested.
- Frontend React components were tested for rendering states accurately (e.g., verifying that passing `locked=true` as a state strictly renders the Lock Screen Modal and never the video player).

8.3 INTEGRATION TESTING

Integration testing verified the interaction between distinct modules.

- **API Endpoints:** Postman was utilized extensively to fire custom HTTP requests to verify that the Express routing logic interfaces correctly with the MongoDB Mongoose models. Tests confirmed that database persistence functions correctly (e.g., submitting an appointment via frontend API call generates a valid MongoDB DB document).

8.4 SYSTEM TESTING

System testing evaluated the fully integrated application. End-to-end flows were tested:

- Simulating a Parent logging in, navigating the dashboard, generating a new child profile, logging out.
- Simulating the Child logging in using the generated PIN, initiating video playback, and holding the connection to verify the server locks the session accurately after X minutes.

8.5 TEST CASES TABLE

Test ID	Test Description	Expected Result	Actual Result	Status
TC01	Parent Registration with valid credentials	Account Created, Data Saved in DB, JWT issued	As Expected	**PASS**

TC02	Login with incorrect password	Application rejection, Error message "Invalid Credentials"	As Expected	**PASS**
TC03	Route Protection: Child attempts to hit `/api/doctor` endpoint	Middleware intercepts request, returns 403 Forbidden	As Expected	**PASS**
TC04	Screen Time Limit Evaluation	Video locks exactly when Frontend Pulse pushes consumed time over the limit	As Expected	**PASS**
TC05	Doctor writing Feedback	System ties feedback string explicitly to the chosen childId and parentId correctly	As Expected	**PASS**

---

9. RESULTS & OUTPUT SCREENS

9.1 DESCRIPTION OF UI/UX

The resulting application features three distinct user experiences:

- **The Parent Dashboard** features a clean, professional, white-and-primary-color layout. It focuses on administrative tasks, forms, and data visualization (grid layouts for displaying medical charts and appointment tables).
- **The Child Interface** is colorful, decluttered, and driven by iconography rather than text, creating an engaging educational environment.
- **The Doctor Interface** uses specialized, high-density table views allowing rapid scanning of patient metrics.

9.2 PROJECT SCREENSHOTS

(Instructions for final document formatting: Ensure you insert the actual screenshots generated by running your React Application locally into these designated placeholder areas before printing).

FIGURE 9.2.1: Login and Role Selection Page

> *Displays the secure entry point where users define their role prior to authentication.*
>
> `[ INSERT SCREENSHOT OF LOGIN UI HERE ]`

FIGURE 9.2.2: Parent Dashboard - Family Overview

> *Displays the primary control panel for the Parent account, illustrating the overview cards of registered children, pending appointments, and system alerts.*
>
> `[ INSERT SCREENSHOT OF PARENT DASHBOARD HERE ]`

FIGURE 9.2.3: Medical Records Viewer

> *Demonstrates the chronological tracking capability showing uploaded bills, prescriptions, and vaccination histories tied to a specific child.*
>
> `[ INSERT SCREENSHOT OF MEDICAL RECORDS PAGE HERE ]`

FIGURE 9.2.4: Child Media Center & Screen Time Lock Implementation

> *Illustrates the child-facing interface, specifically highlighting the moment the backend algorithm intervenes and overrides the DOM to present the "Time's Up" Lock screen.*
>
> `[ INSERT SCREENSHOT OF CHILD VIDEO / LOCK SCREEN HERE ]`

FIGURE 9.2.5: Doctor Portal & Clinical Feedback Mechanism

> *Shows the Doctor's isolated environment displaying their scheduled patients, and the specialized form utilized to inject permanent feedback into the parent's health tracking instance.*
>
> `[ INSERT SCREENSHOT OF DOCTOR PORTAL UI HERE ]`

---

10. CONCLUSION & FUTURE SCOPE

10.1 CONCLUSION

The Baby Care Management Application represents a highly effective modernization of early childhood care coordination. By strategically unifying diverse requirements—secure medical tracking, appointment scheduling, and strict digital media regulation—the project successfully alleviates the massive organizational burden traditionally placed on parents.

The implementation of the MERN stack architecture proved exceptionally capable of handling complex asynchronous requirements like real-time screen time polling while maintaining a rapidly responsive UI. The strict enforcement of cryptographic security (Bcrypt, JWT) and Role-Based Access Control successfully ensures that highly sensitive medical data and clinical feedback are heavily guarded and accessible only by strictly authorized entities. Ultimately, this application not only acts as an organizational utility but as a cohesive ecosystem that genuinely contributes to the optimized developmental journey of a child.

10.2 FUTURE ENHANCEMENTS

While the current scope fulfills core objectives, the system architecture is designed to be highly scalable, allowing for the following integrations in future iterations:

1. **IoT & Wearable Integration:** Expanding API endpoints to ingest real-time biometric data (heart rate, sleep cycles, GPS location) straight from popular children's smartwatches to provide actionable health insight without manual data entry.
2. **Artificial Intelligence Data Analysis:** Leveraging machine learning algorithms on the collected growth and task data. AI could compare an individual child's metrics against global pediatric growth charts to flag early signs of developmental delays, or suggest tailored educational content based on their interaction history.

3. **Cross-Platform Mobile App Deployment:** Porting the React frontend components into React Native to create dedicated iOS and Android applications. This would vastly improve background capabilities, allowing native push notifications to remind parents of upcoming vaccinations and doctor appointments instantly.
4. **Automated Content Filtering AI:** Upgrading the Media Center module to utilize Computer Vision AI to actively scan videos in real-time, automatically censoring or locking content bypassing manual curation standards.

---

11. BIBLIOGRAPHY

1. Banks, A., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media.
    - Used for structural understanding of modern React Hook integrations and state handling.
2. Mardan, A. (2018). *Pro Express.js: Master Express.js: The Node.js Framework For Your Web Development*. Apress.
    - Reference material for constructing robust, scaled RESTful API architectures using Node and Express.
3. Node.js Foundation. (2024). *Official Node.js API Documentation*. Retrieved from https://nodejs.org/en/docs/
4. MongoDB Inc. (2024). *MongoDB Manual and Query Language Reference*. Retrieved from https://www.mongodb.com/docs/
5. Internet Engineering Task Force (IETF). (2015). *RFC 7519: JSON Web Token (JWT)*. Authored by M. Jones, et al. Retrieved from https://jwt.io/
6. Meta Platforms. (2024). *React Official Documentation*. Retrieved from https://react.dev/
7. Vite.js Team. (2024). *Vite: Next Generation Frontend Tooling*. Retrieved from https://vitejs.dev/guide/